LESSON 3

シーン間共有データ

目標：シーンを切り替えても、スコアや設定を引き継げるようにする

導入

シーンを切り替えると値が消える

GameScene

score = 1200

LoadScene



ResultScene

score = ？

引き継ぎたいもの①

スコア・残機・プレイ時間など、ゲームの結果。

引き継ぎたいもの②

選んだキャラクター・ステージ・難易度。

引き継ぎたいもの③

音量・ボタン配置などの設定。

悪い例

前のシーンのオブジェクトを探す

C# ResultView.cs

```
// ResultScene の ResultView.cs
void Start()
{
    // 前のシーンの ScoreCounter を探す
    ScoreCounter counter = FindAnyObjectByType<ScoreCounter>();
    scoreText.text = counter.Score.ToString();   // ← ここでエラー
}
```

⚠ ScoreCounter は GameScene と一緒に消えているので、見つからずに **null** が返る。
そのまま使うと **NullReferenceException** になる。

比較

データを共有する4つの方法

方法	いつ消える？	特徴	向いているもの
static 変数	ゲームを閉じたとき	いちばん手軽	スコア・選んだステージ
PersistentSingleton	ゲームを閉じたとき	処理も一緒に持てる	ゲーム進行の管理
ScriptableObject	アセットとして残る	インスペクタで編集できる	敵の強さなどの設定値
PlayerPrefs	消えない（保存される）	端末に保存する	ハイスコア・音量設定

POINT

上の3つは**実行中だけ**の共有。今回の GameDataHolder は **PersistentSingleton** を使う。ゲームを閉じても残したいものは **PlayerPrefs** で保存する。

方法①

static 変数で共有する

C# GameSession.cs

```
public static class GameSession
{
    public static int score;
    public static int selectedStage;
}

// GameScene で
GameSession.score += 100;

// ResultScene で
scoreText.text = GameSession.score.ToString();
```

POINT static の変数はインスタンスではなく**クラスに1つ**。シーンを切り替えても値が残る。

提供スクリプトの解説

GameDataHolder 全体

C# GameDataHolder.cs

```
using UnityEngine;

public class GameDataHolder : PersistentSingleton<GameDataHolder>
{
    [SerializeField] public int score;


    /// <summary>
    /// ゲーム起動時に生成
    /// </summary>
    [RuntimeInitializeOnLoadMethod]
    private static void Init()
    {
        GameDataHolder pref = Resources.Load<GameDataHolder>("Singletons/GameDataHolder");
        if (pref != null) Instantiate(pref);
    }
}
```

POINT

SceneLoadManager・AudioManager と同じ形。ちがうのは **score を持っているだけ**。シーンを切り替えても GameDataHolder ごと残るので、score も残る。

提供スクリプト

GameDataHolderDemo.cs 全文

C# GameDataHolderDemo.cs

```
using UnityEngine;
using TMPro;

public class GameDataHolderDemo : MonoBehaviour
{
    TextMeshProUGUI tmp;

    private void Start()
    {
        tmp = GetComponent<TextMeshProUGUI>();

        tmp.text = GameDataHolder.Instance.score.ToString();
    }

    void Update()
    {
        if(Input.GetKeyDown(KeyCode.DownArrow))
        {
            GameDataHolder.Instance.score++;
            tmp.text = GameDataHolder.Instance.score.ToString();
        }
    }
}
```

実装

GameDataHolderDemo ① スコアを表示する

C# GameDataHolderDemo.cs

```cs
using UnityEngine;
using TMPro;

public class GameDataHolderDemo : MonoBehaviour
{
    TextMeshProUGUI tmp;

    private void Start()
    {
        tmp = GetComponent<TextMeshProUGUI>();

        tmp.text = GameDataHolder.Instance.score.ToString();
    }
```

using TMPro;

TextMeshPro の文字を使うために必要。

GetComponent<TextMeshProUGUI>()

同じオブジェクトの Text（TextMeshPro）を取得する。

GameDataHolder.Instance.score

どのシーンからでも、同じ score を読める。

ToString()

int を文字列に変えて表示する。

提供スクリプトの解説

GameDataHolderDemo ② ↓キーでスコアを増やす

C# GameDataHolderDemo.cs（続き）

```
15      void Update()
16      {
17          if(Input.GetKeyDown(KeyCode.DownArrow))
18          {
19              GameDataHolder.Instance.score++;
20              tmp.text = GameDataHolder.Instance.score.ToString();
21          }
22      }
23  }
```

score++

どこからでも書き換えられる

public なので、ほかのスクリプトから直接変更できる。

tmp.text = ...

表示も更新する

値を変えただけでは文字は変わらない。毎回 text に入れ直す。

Input.GetKeyDown

古い入力の書き方

Unity 6 では設定を Both にしないとエラーになる（準備で設定）。

注意

public は誰でも書き換えられる

C# 危ない例と改良例

```
// どのスクリプトからでも、自由に書き換えられてしまう
GameDataHolder.Instance.score = -999;

// 改良例：読むのは誰でも、変えるのはメソッド経由だけ
public int Score { get; private set; }

public void AddScore(int amount)
{
    Score += amount;
}
```

private set

外からは読むだけ。変えるのは AddScore などのメソッドからだけ。

リセットを忘れない

タイトルに戻ったら score を 0 に戻す。戻さないと前回の値が残る。

準備

Unity で設定する

手順	作業	ポイント
1	Hierarchy で Create Empty、名前を GameDataHolder にする	Resources.Load の名前と同じにする
2	GameDataHolder スクリプトをアタッチ	Score の初期値は Inspector で決められる
3	Resources/Singletons/ にドラッグしてプレハブ化	ほかのマネージャーと同じ場所
4	シーン上の GameDataHolder を削除	起動時に自動で生成される
5	2つのシーンに Text (TextMeshPro) を置き、GameDataHolderDemo をアタッチ	TextMeshProUGUI と同じオブジェクトに付ける
6	Project Settings ＞ Player ＞ Active Input Handling を Both にする	Input.GetKeyDown を使うため

確認

Unity で確かめる

手順	やること	確認すること
1	↓キーを何回か押す	画面のスコアが増える
2	シーンを切り替える	同じスコアが表示される
3	ゲームを止めて、もう一度再生する	スコアが 0 に戻る（保存はされていない）
4	2つ目のシーンから直接再生する	エラーにならない（自動で生成されるため）

POINT 手順3のとおり、GameDataHolder は**実行中だけ**の共有。ゲームを閉じても残すには、次の PlayerPrefs で保存する。

方法④

PlayerPrefs で保存する

C# ハイスコアの保存

```
const string HIGH_SCORE_KEY = "HighScore";

// 保存
PlayerPrefs.SetInt(HIGH_SCORE_KEY, score);
PlayerPrefs.Save();

// 読み込み（保存がなければ 0）
int highScore = PlayerPrefs.GetInt(HIGH_SCORE_KEY, 0);
```

SetInt / SetFloat / SetString

保存できる型

int・float・string の3種類。bool は 0/1 の int で保存する。

const string ..._KEY

キーは定数にする

"HighScore" と毎回書くと打ち間違いに気づけない。

注意

ScriptableObject の落とし穴

エディタで実行したとき

- 実行中に値を書き換えると、**止めても値が残る**
- アセットそのものを書き換えているため
- 次に再生したとき、前回の値から始まる

ビルドしたゲームでは

- ゲームを閉じると**元の値に戻る**
- 保存の代わりには使えない
- エディタと動きがちがう

⚠ **ScriptableObject は「変わらない設定値」に使い、実行中の値は入れない**

演習

データを引き継ごう

1 **リザルトにスコアを表示する**

GameDataHolderDemo を参考に、別のシーンでスコアを表示しよう。

2 **タイトルに戻ったら 0 にする**

タイトル画面が始まったときに、score を 0 に戻そう。

3 **ハイスコアを保存する**

PlayerPrefs を使って、ゲームを閉じてもハイスコアが残るようにしよう。

4 **発展：書き換えの入口を絞る**

score を private set のプロパティに変えて、AddScore と ResetScore を作ろう。

LESSON 4

PADコンフィグ

目標：プレイヤーがボタンの割り当てを変えられるようにする

導入

PADコンフィグとは

「ジャンプ」「攻撃」などの操作に、**どのボタンを使うかをプレイヤーが決められる**機能。

遊びやすさ

手の大きさや持ち方、慣れているゲームに合わせて変えられる。

いろいろなコントローラー

Xbox・PlayStation・Switch など、ボタンの配置がちがっても対応できる。

アクセシビリティ

片手で遊ぶ人など、押しにくいボタンを避けられる。

悪い例

キーを直接チェックする

C# PlayerJump.cs

```
void Update()
{
    if (Keyboard.current.spaceKey.wasPressedThisFrame
        || Gamepad.current.buttonSouth.wasPressedThisFrame)
    {
        Jump();
    }
}
```

① 変えられない

Space と下ボタンがコードに書かれているので、プレイヤーが変更できない。

② パッドがないとエラー

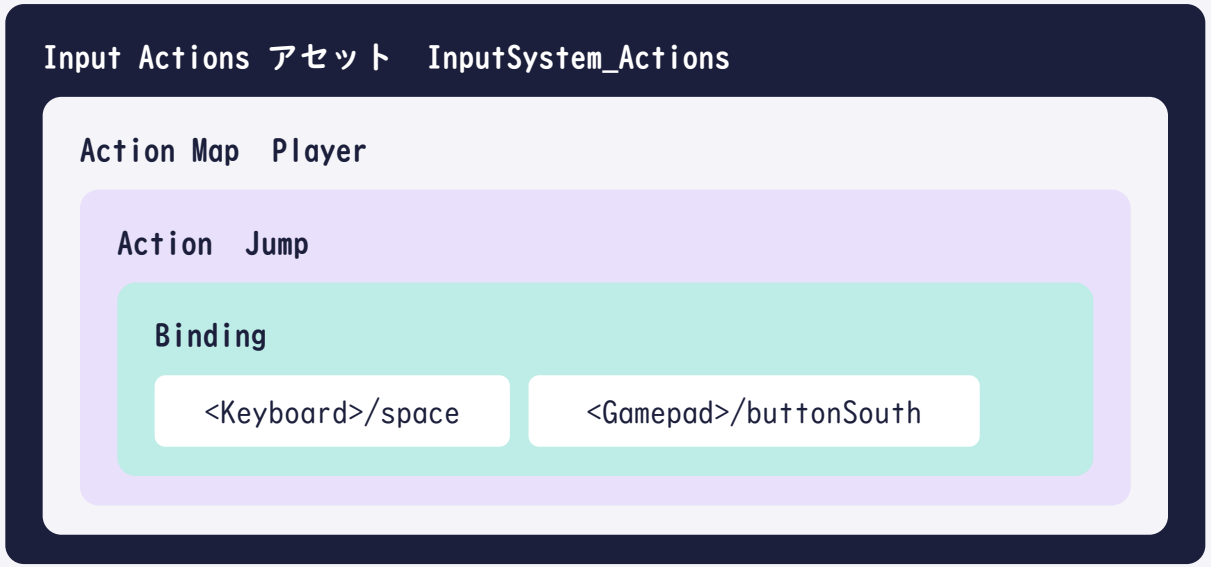
パッドが未接続だと Gamepad.current が null になる。

③ 条件が増え続ける

対応したいキーが増えるたびに || が増えていく。

基本

Input System の構成



Action Map
場面ごとの操作のまとまり（Player / UI）

Action
ゲームでの意味（ジャンプ・攻撃）

Binding
実際のキーやボタン

POINT　PADコンフィグは、**Action はそのままで Binding だけを差し替える**しくみ。

基本

パッドのボタンは「位置」で呼ぶ

Input System の名前	位置	Xbox	PlayStation	Switch
buttonSouth	下	A	×	B
buttonEast	右	B	○	A
buttonWest	左	X	□	Y
buttonNorth	上	Y	△	X
leftShoulder / rightShoulder	LB・RB	LB / RB	L1 / R1	L / R

POINT 同じ「下のボタン」でも、Xbox では A、Switch では B。**画面に表示するときは名前を変換する**必要がある。

基本

Action を使って書く

C# PlayerJump.cs

```
[SerializeField] InputActionReference jumpAction;

void OnEnable()  { jumpAction.action.Enable(); }
void OnDisable() { jumpAction.action.Disable(); }

void Update()
{
    if (jumpAction.action.WasPressedThisFrame())
    {
        Jump();
    }
}
```

POINT 「Space が押されたか」ではなく**「ジャンプが押されたか」**を聞く。どのボタンかは Input Actions 側で決まる。

基本

リバインド（割り当ての変更）

C# 割り当て変更の例

```
jumpAction.action.Disable();   // 変更中は止めておく

jumpAction.action.PerformInteractiveRebinding(0)
    .WithControlsExcluding("<Mouse>")
    .WithCancelingThrough("<Keyboard>/escape")
    .OnComplete(operation =>
    {
        operation.Dispose();          // 後片付け
        jumpAction.action.Enable();
    })
    .Start();
```

PerformInteractiveRebinding(0)

次に押したボタンを、0番目の Binding に登録する

WithCancelingThrough

Esc を押したら変更をやめる

OnComplete

登録が終わったときの処理

基本

割り当てを保存する

C# 保存と読み込み

```
const string BINDINGS_KEY = "InputBindings";

// 保存
string json = inputActions.SaveBindingOverridesAsJson();
PlayerPrefs.SetString(BINDINGS_KEY, json);

// 読み込み（ゲーム起動時）
string saved = PlayerPrefs.GetString(BINDINGS_KEY, "");
inputActions.LoadBindingOverridesFromJson(saved);
```

SaveBindingOverridesAsJson()

変更した分だけ文字列にする

前回の授業の PlayerPrefs で保存できる。

RemoveAllBindingOverrides()

初期設定に戻す

「デフォルトに戻す」ボタンに使う。

GetBindingDisplayString()

画面に表示する

「Space」「A」など、今のボタン名を取り出す。

提供スクリプトの解説

PADコンフィグ ① 全体の構成

スクリプト受領後に差し替え

C# （提供スクリプト）.cs

提供スクリプトのコードが入ります

（長い場合は複数ページに分割）

このページで説明すること

- Input Actions の持ち方
- 変更できる Action の一覧
- 起動時の読み込み
- 保存のキーの定数

提供スクリプトの解説

PADコンフィグ ② リバインドと UI

スクリプト受領後に差し替え

C# （提供スクリプト）.cs（続き）

提供スクリプトのコードが入ります

（長い場合は複数ページに分割）

このページで説明すること

- ボタンを押したときのリバインド開始
- 変更中の表示（「ボタンを押してください」）
- ボタン名の表示の更新
- 初期設定に戻す処理

準備

Unity で設定する

手順	作業	確認すること
1	Edit ＞ Project Settings ＞ Player ＞ Active Input Handling	Input System Package（New）
2	InputSystem_Actions を開く	Unity 6 では最初から用意されている
3	Player マップに Jump・Attack などを確認	Keyboard と Gamepad の Binding がある
4	コンフィグ画面の UI を作る	Action ごとにボタンとテキスト
5	PADコンフィグのスクリプトをアタッチ	Action を Inspector で設定
6	パッドをつないで確認	変更したボタンでジャンプできる

POINT 手順は提供スクリプトの仕様に合わせて調整する。

演習

PADコンフィグを作ろう

1 **ジャンプを Action で書き直す**

キーを直接チェックしているコードを、Action を使う形に直そう。

2 **ジャンプのボタンを変更する**

コンフィグ画面で、ジャンプのボタンを好きなボタンに変えられるようにしよう。

3 **保存して確かめる**

ゲームを止めて再生し直しても、変更したボタンが残っているか確認しよう。

4 **発展：同じボタンの重複を防ぐ**

ジャンプと攻撃が同じボタンにならないようにしよう。

ヒント：OnPotentialMatch で他の Binding と比べる